학습 교안

머신러닝 입문자를 위한 기초 수학

기호 · 합계(Σ) · 평균 · 1차/2차 함수 · 미분 · 편미분 · 합성함수의 미분

항목	내용
원본 영상	테디노트 TeddyNote, 「#머신러닝 #딥러닝 입문자를 위한 초스피드 기초 #수학」(13분 45초, 2020.08.29)
영상 주소	https://www.youtube.com/watch?v=vS51prw_yfw
대상	수학·통계를 오래 쉬었거나 비전공자로서 머신러닝/딥러닝을 처음 시작하는 학생
학습 시간	영상 시청 15분 + 교안 정리 및 실습 약 2시간
준비물	Python 3, numpy, sympy, matplotlib (실습 코드 폴더의 README 참고)

목차

0장 들어가며 . . . 3

1장 기호 읽기: 실제값 · 예측값 · 평균값 . . . 4

2장 함수 표기: y = f(x) . . . 6

3장 합계 기호 Σ (Summation) . . . 7

4장 평균과 Σ 의 성질 . . . 9

5장 1차 함수: y = wx + b . . . 11

6장 2차 함수: 아래로 볼록한 그래프 . . . 13

7장 미분 (Derivative) . . . 14

8장 편미분 (Partial Derivative) . . . 16

9장 합성함수의 미분 (연쇄법칙) . . . 18

10장 머신러닝에 연결하기 (보충) . . . 20

부록 A 확인 문제 정답 및 해설 . . . 23

부록 B 기호 · 공식 한눈에 보기 . . . 25

CHAPTER 0

들어가며

머신러닝·딥러닝 책이나 논문을 펼치면 처음부터 $\hat{y}$, Σ, ∂ 같은 기호가 쏟아집니다. 어려운 수학을 깊게 알 필요는 없지만, **기호를 읽고 간단한 미분을 손으로 할 수 있는 정도**는 되어야 경사하강법(Gradient Descent)이나 최소제곱법(OLS) 같은 핵심 알고리즘을 이해할 수 있습니다.

이 교안은 영상의 순서를 그대로 따라가며, 각 개념을 ① **수식** → ② **그림** → ③ **파이썬 코드**로 세 번 확인합니다. 마지막 10장에서는 영상에서 배운 도구를 모두 모아 실제 머신러닝의 학습 과정(손실 함수의 미분과 경사하강법)을 직접 계산해 봅니다.

교안 구성

장	주제	영상 구간	한 줄 요약
1	기호 읽기	00:39~01:25	y 실제값, $\hat{y}$ 예측값, $\bar{y}$ 평균값
2	함수 표기	01:25~01:45	y = f(x) 는 'x 를 넣으면 y 가 나온다'
3	합계 Σ	01:45~02:32	Σ 는 for 반복문으로 더하기
4	평균과 Σ 의 성질	02:32~04:30	$\Sigma x = n \cdot \bar{x}$, $\Sigma b = n \cdot b$
5	1차 함수 wx + b	04:30~05:50	w 는 기울기(가중치), b 는 높낮이(편향)
6	2차 함수	05:50~06:10	아래로 볼록한 그릇 모양, 바닥이 최솟값
7	미분	06:10~08:20	한 점에서의 기울기, $x^n \rightarrow n \cdot x^{n-1}$
8	편미분	08:20~09:47	나머지 변수는 상수 취급
9	합성함수의 미분	09:47~12:44	겉 미분 × 속 미분 (연쇄법칙)
10	머신러닝에 연결하기 (보충)	12:44~끝	MSE 미분 → 경사하강법 직접 구현

공부하는 방법

1) 각 장의 **영상 구간**을 먼저 보고, 2) 교안의 설명과 그림으로 정리한 뒤, 3) 실습 코드를 직접 실행해 결과를 확인하세요. 4) 장 끝의 **확인 문제**를 손으로 풀고 부록 A 의 정답과 맞춰 봅니다. 10장은 영상에 없는 보충 내용이지만, 영상 끝에서 "앞으로 이 지식이 필요하다"고 한 바로 그 부분입니다.

핵심 포인트

- 수식은 '읽는 법'부터 익히면 절반은 끝납니다. 기호를 소리 내어 읽어 보세요. (예: $\hat{y}$ → "와이 햇", ∂ → "라운드")
- 코드로 숫자를 넣어 확인하면 수식이 훨씬 덜 무섭습니다. 모든 실습 결과는 실제로 실행한 값입니다.

CHAPTER 1

기호 읽기: 실제값 · 예측값 · 평균값

영상 구간 **00:39~01:25** 앞으로 계속 나올 세 가지 기호

학습 목표

- y, ŷ, ȳ 가 각각 무엇을 뜻하는지 구분할 수 있다.
- 오차 y - ŷ 의 의미를 설명할 수 있다.

머신러닝 수식에서 가장 많이 보게 될 세 가지 기호입니다. 모두 **y 위에 무엇이 붙었느냐**로 구분합니다.

기호	읽는 법	뜻	예시
y	와이	**실제값** - 데이터에 들어 있는 정답	실제 집값 3억
$\hat{y}$	와이 햇(hat)	**예측값** - 모델이 예측한 값 (모자 ^ 를 씀)	모델이 예측한 집값 2.8억
$\bar{y}$	와이 바(bar)	**평균값** - 실제값 y 들의 평균 (막대 - 를 씀)	전체 집값의 평균 2.5억

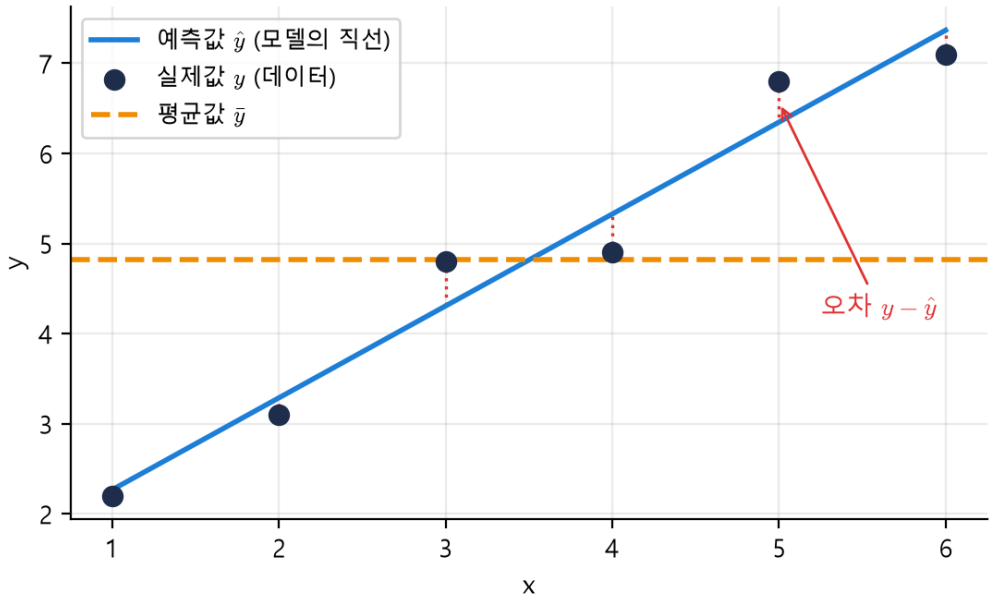


그림 1-1. 점은 실제값 y, 파란 직선은 예측값 ŷ, 주황 점선은 평균 ȳ. 빨간 점선이 오차 y - ŷ

모델이 잘 학습되었다는 것은 **y 와 ŷ 의 차이(오차)가 작다**는 뜻입니다. 머신러닝의 학습이란 결국 이 오차를 줄이는 방향으로 모델을 고쳐 나가는 과정입니다. (10장에서 직접 해 봅니다.)

실습 코드 0 - 준비

```
import numpy as np
import sympy as sp
np.set_printoptions(precision=4, suppress=True)
print("numpy", np.__version__, "/ sympy", sp.__version__)
```

실행 결과

```
numpy 2.5.4 / sympy 1.14.0
```

실습 코드 1-1 - 실제값 · 예측값 · 평균값

```
y     = np.array([3.0, 5.0, 7.0, 9.0])   # 실제값 y (정답 데이터)
y_hat = np.array([2.5, 5.5, 6.5, 9.5])   # 모델이 예측한 값 y hat
y_bar = y.mean()                          # 실제값의 평균 y bar

print("실제값 y      :", y)
print("예측값 y_hat  :", y_hat)
print("평균값 y_bar  :", y_bar)
print("오차 y - y_hat:", y - y_hat)
```

실행 결과

```
실제값 y      : [3. 5. 7. 9.]
예측값 y_hat  : [2.5 5.5 6.5 9.5]
평균값 y_bar  : 6.0
오차 y - y_hat: [ 0.5 -0.5  0.5 -0.5]
```

핵심 포인트

• y = 정답, ŷ = 모델의 답, ȳ = 정답들의 평균.
• 오차 = y - ŷ. 학습은 이 오차를 줄이는 과정이다.

확인 문제

Q1. 어떤 모델이 키 170cm 인 학생의 키를 168cm 로 예측했다. y, ŷ, 오차 y - ŷ 를 각각 쓰시오.
Q2. y = [4, 6, 8] 일 때 ȳ 를 구하시오.

CHAPTER 2

함수 표기: y = f(x)

영상 구간 **01:25~01:45** y = f(x), 1차 함수 y = ax + b

학습 목표

- y = f(x) 표기를 읽고, 함수가 '입력 → 출력' 규칙임을 설명할 수 있다.

함수(function)는 입력 x 를 넣으면 정해진 규칙에 따라 출력 y 가 하나 나오는 상자입니다. 그 규칙에 이름을 붙여 f 라고 부르고 **y = f(x)** ("와이는 에프 엑스") 로 씁니다.

$$y = f(x) \quad \longrightarrow \quad y = ax + b \iff f(x) = ax + b$$

즉 **y = ax + b** 와 **f(x) = ax + b** 는 같은 식입니다. 파이썬으로 보면 함수 정의와 똑같습니다.

```
def f(x):
    return a * x + b     # f(x) = ax + b

y = f(3)                 # x = 3 을 넣으면 y 가 나온다
```

머신러닝 모델도 결국 하나의 큰 함수입니다. 입력(사진, 문장, 숫자 데이터)을 넣으면 예측값 ŷ 가 나옵니다. 그래서 ŷ = f(x) 라고 쓰기도 합니다.

핵심 포인트

- f(x) 는 "x 를 넣은 함수 f 의 값". 곱셈 f × x 가 아니다.
- 모델 = 함수, 모델의 출력 = ŷ = f(x).

확인 문제

Q1. f(x) = 3x - 2 일 때 f(4) 를 구하시오.
Q2. y = 5x + 1 을 f(x) 표기로 바꿔 쓰시오.

CHAPTER 3

합계 기호 Σ (Summation)

영상 구간 **01:45~02:32** 시그마 읽는 법, 반복문으로 이해하기

학습 목표
- Σ 기호의 위·아래 숫자의 의미를 읽을 수 있다.
- Σ 를 파이썬 for 반복문으로 바꿔 쓸 수 있다.

Σ 는 그리스 문자 시그마로, **모두 더하라(Summation)**는 뜻입니다.

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_{n-1} + x_n$$

위치	읽는 법	의미
아래 i = 1	i 는 1 부터	어디서부터 더할지 (시작 번호)
위 n	n 까지	어디까지 더할지 (끝 번호, 보통 데이터 개수)
오른쪽 x_i	x 아이	i 번째 데이터. i 가 1, 2, ..., n 으로 바뀌며 하나씩 대입

개발자라면 **for 반복문**을 떠올리면 됩니다. i 가 1 부터 n 까지 1 씩 증가하면서 x_i 를 하나씩 누적해 더합니다. 단, 수학은 보통 1 번부터, 파이썬 인덱스는 0 번부터 센다는 점만 주의하세요.

실습 코드 3-1 - Σ 는 반복문

```
x = [2, 4, 6, 8, 10]          # x1 ~ x5  (n = 5)

total = 0
for i in range(len(x)):       # 수학의 i = 1..n  ->  파이썬은 0..n-1
    total += x[i]

print("반복문으로 더한 값:", total)
print("sum() 결과        :", sum(x))
print("np.sum() 결과     :", np.sum(x))
```

실행 결과

```
반복문으로 더한 값: 30
sum() 결과        : 30
np.sum() 결과     : 30
```

위아래 범위를 생략하고 **Σx** 처럼 쓰는 경우도 많습니다. 이때는 "가지고 있는 데이터 전부를 더한다"는 뜻으로 읽으면 됩니다.

핵심 포인트
- Σ = 더하기 반복문. 아래는 시작, 위는 끝.
- 범위가 생략된 Σx 는 '전체 합'.

확인 문제

Q1. x = [1, 3, 5, 7] 일 때 다음 값을 구하시오.

$$\sum_{i=1}^{4} x_i \qquad \sum_{i=2}^{3} x_i$$

Q2. 1 + 2 + 3 + ... + 100 을 Σ 기호로 쓰시오.

CHAPTER 4

평균과 Σ 의 성질

영상 구간 **02:32~04:30** 평균 = (1/n)·Σ, Σx = n·x̄, Σb = n·b

학습 목표
- 평균을 Σ 로 표현할 수 있다.
- Σx = n·x̄ 와 Σb = n·b (b 는 상수) 를 설명할 수 있다.

4.1 평균 (Mean, Average)

평균은 "모두 더한 뒤 개수로 나눈 값"입니다. 모두 더하는 것은 Σ 이므로, 앞에 **1/n** 만 곱하면 평균이 됩니다.

$$\frac{1}{n}\sum_{i=1}^{n} x_i \;=\; \frac{x_1 + x_2 + \cdots + x_n}{n} \;=\; \bar{x}$$

1장의 ȳ 와 같은 방식으로, x 의 평균은 x̄ (엑스 바) 라고 씁니다.

4.2 성질 ① 합계 = 개수 × 평균

위 식의 양쪽에 n 을 곱하면 1/n 이 사라지면서 아주 자주 쓰는 공식이 나옵니다.

$$\frac{1}{n}\sum x = \bar{x} \quad\Longrightarrow\quad \sum x = n \times \bar{x}$$

주의
- 영상(03:45)의 판서에는 x̄ 아래에 i 가 붙어 있지만($\bar{x}_i$), 평균은 데이터 전체에서 계산한 **하나의 값**이므로 i 를 붙이지 않습니다. 올바른 표기는 $\Sigma x_i = n\cdot\bar{x}$ 입니다.

4.3 성질 ② 상수의 합 = n × 상수

b 가 2 나 100 처럼 변하지 않는 **상수**라면, Σb 는 b 를 n 번 더한 것이므로 n·b 입니다.

$$\sum_{i=1}^{n} b \;=\; b + b + \cdots + b \;\;(n \text{ times}) \;=\; n \times b$$

4.4 (보충) 성질 ③ 상수는 Σ 밖으로

영상에는 없지만 10장에서 꼭 필요한 성질입니다. 모든 항에 같은 수 c 가 곱해져 있으면 묶어서 밖으로 꺼낼 수 있습니다. 또, 더하기는 따로따로 Σ 해도 됩니다.

$$\sum c\,x_i = c\sum x_i \qquad\qquad \sum (x_i + y_i) = \sum x_i + \sum y_i$$

실습 코드 4-1 - 평균과 Σ 의 성질 확인

```
n = len(x)
x_bar = sum(x) / n                          # (1/n) * Σx
print("평균 x_bar =", x_bar, " / np.mean =", np.mean(x))

# 규칙 1)  Σx = n * x_bar
print("Σx =", sum(x), "   n * x_bar =", n * x_bar)

# 규칙 2)  상수 b 를 n 번 더하면  n * b
b = 7
print("Σb =", sum(b for _ in range(n)), "   n * b =", n * b)

# 규칙 3)  상수 c 는 Σ 밖으로 꺼낼 수 있다
c = 3
print("Σ(c*x) =", sum(c * xi for xi in x), "   c * Σx =", c * sum(x))
```

실행 결과

```
평균 x_bar = 6.0  / np.mean = 6.0
Σx = 30    n * x_bar = 30.0
Σb = 35    n * b = 35
Σ(c*x) = 90    c * Σx = 90
```

핵심 포인트

- 평균 $\bar{x}$ = (1/n)Σx ⇔ Σx = n·$\bar{x}$
- Σb = n·b (상수를 n 번 더함), Σcx = cΣx (상수는 밖으로)

확인 문제

Q1. x = [10, 20, 30, 40, 50] 일 때 $\bar{x}$ 와 Σx 를 구하고, Σx = n·$\bar{x}$ 가 성립하는지 확인하시오.
Q2. n = 8 일 때 Σ 5 (i = 1 부터 8 까지) 의 값은?
Q3. Σx = 120, n = 6 이면 $\bar{x}$ 는?

CHAPTER 5

1차 함수: y = wx + b

영상 구간 **04:30~05:50** 기울기·절편 → 가중치(weight)·편향(bias)

학습 목표
- 1차 함수의 기울기와 절편을 머신러닝 용어(w, b)로 바꿔 말할 수 있다.
- w 와 b 가 바뀔 때 직선이 어떻게 움직이는지 설명할 수 있다.

중학교에서 배운 1차 함수 y = ax + b 에서 a 는 **기울기**, b 는 **y 절편**이었습니다. 머신러닝에서는 같은 식을 문자만 바꿔 씁니다.

$$y = ax + b \quad \Longrightarrow \quad \hat{y} = wx + b$$

중학교 수학	머신러닝	영어	역할
a (기울기)	w (가중치)	weight	입력 x 가 결과에 얼마나 영향을 주는지. 직선의 기울기
b (y 절편)	b (편향)	bias	입력과 상관없이 더해지는 기본값. 직선의 높낮이

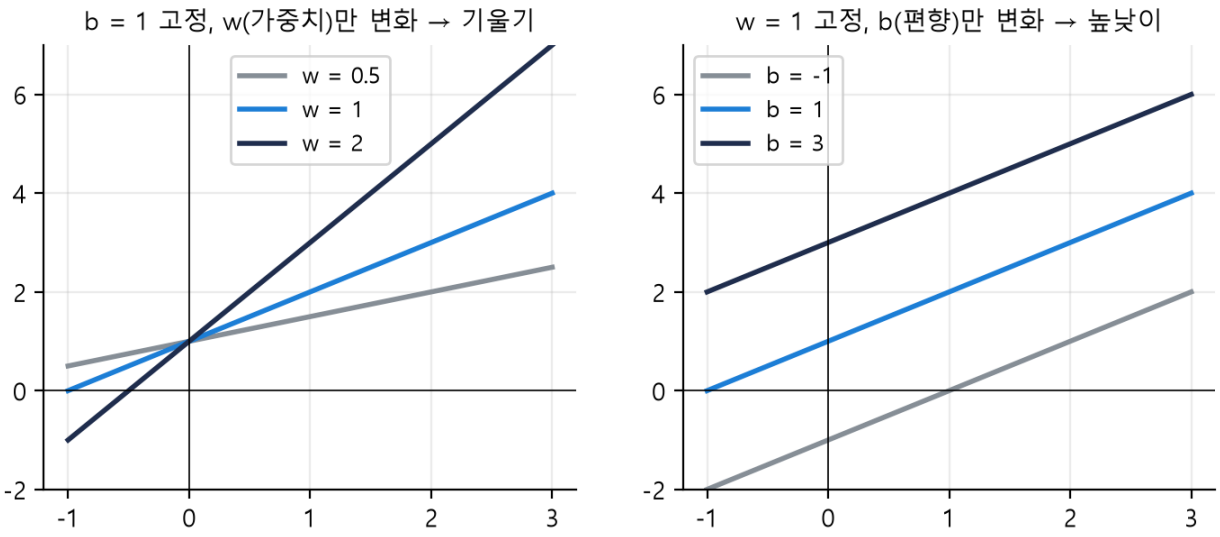


그림 5-1. 왼쪽: w 를 바꾸면 기울기가 변한다. 오른쪽: b 를 바꾸면 직선이 위아래로 평행이동한다

모델을 **학습**시킨다는 것은 데이터를 가장 잘 설명하는 **w 와 b 값을 찾는 것**입니다. 이렇게 학습으로 찾아내는 값을 **파라미터(parameter)**라고 부릅니다.

실습 코드 5-1 - w, b 를 바꿔 보기

```
def f(x, w, b):
    return w * x + b                    # y = wx + b

xs = np.array([0, 1, 2, 3])
print("w=2, b=1 :", f(xs, 2, 1))
print("w=3, b=1 :", f(xs, 3, 1), " <- w 증가: 기울기가 가팔라짐")
print("w=2, b=5 :", f(xs, 2, 5), " <- b 증가: 위로 평행이동")
```

실행 결과

```
w=2, b=1 : [1 3 5 7]
w=3, b=1 : [ 1  4  7 10]  <- w 증가: 기울기가 가팔라짐
w=2, b=5 : [ 5  7  9 11]  <- b 증가: 위로 평행이동
```

핵심 포인트

- ŷ = wx + b: w 는 가중치(기울기), b 는 편향(높낮이).
- 학습 = 좋은 w, b 를 찾는 것.

확인 문제

Q1. ŷ = 3x + 2 에서 가중치와 편향은 각각 얼마인가? x = 4 일 때 ŷ 는?
Q2. 직선을 기울기는 그대로 두고 위로 3 만큼 올리려면 w, b 중 무엇을 어떻게 바꿔야 하는가?

CHAPTER 6

2차 함수: 아래로 볼록한 그래프

영상 구간 **05:50~06:10** y = x², 볼록(convex) 함수

학습 목표
- 2차 함수 그래프의 모양을 그릴 수 있다.
- 아래로 볼록한 함수에서 '최솟값(바닥)'의 의미를 이해한다.

가장 기본적인 2차 함수는 **y = x²** 입니다. 그래프는 아래로 볼록한 그릇 모양이고, 이런 함수를 **볼록 함수 (convex function)**라고 부릅니다.

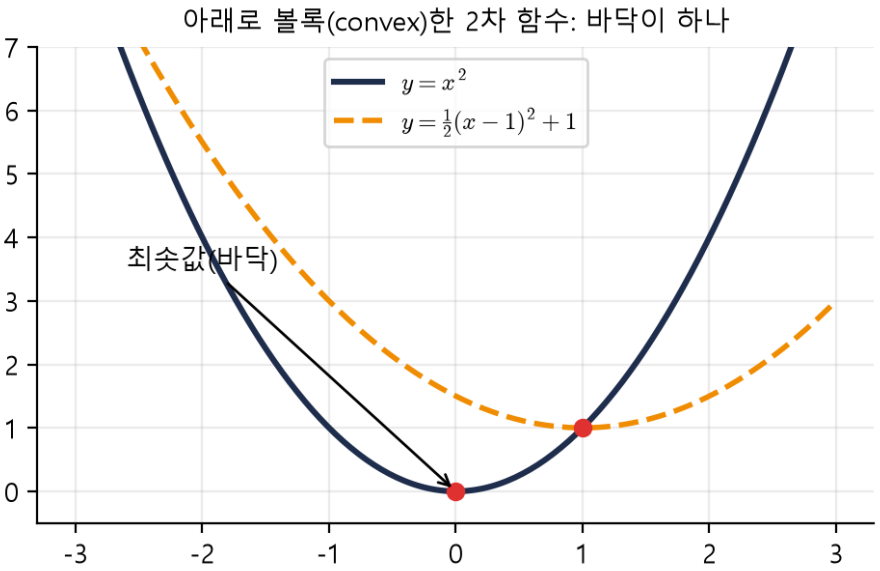


그림 6-1. 아래로 볼록한 2차 함수는 바닥(최솟값)이 딱 하나 있다

왜 머신러닝에서 2차 함수가 중요할까요? 10장에서 보겠지만, 오차를 제곱해서 평균 낸 **손실 함수(MSE)**를 w 에 대해 그려 보면 바로 이런 그릇 모양이 됩니다. 그러면 학습은 "그릇의 **바닥**을 찾아 내려가는 일"이 되고, 바닥으로 가는 방향을 알려 주는 도구가 다음 장의 **미분**입니다.

핵심 포인트
- y = x² 는 아래로 볼록(convex), 최솟값이 하나.
- 손실 함수의 바닥 = 가장 좋은 파라미터.

확인 문제

Q1. y = x² 의 최솟값과 그때의 x 를 구하시오.

Q2. y = (x - 3)² + 1 의 최솟값과 그때의 x 를 구하시오.

CHAPTER 7

미분 (Derivative)

영상 구간 **06:10~08:20** 순간 변화량 = 한 점에서의 기울기, 기본 미분 3가지

학습 목표
- 미분을 '한 점에서의 접선의 기울기'로 설명할 수 있다.
- 상수, 1차식, x^n 을 미분할 수 있다.

미분은 **순간 변화량**, 쉽게 말해 **그래프 위 한 점에서의 기울기**입니다. 그 점에 딱 맞닿는 직선(접선)을 그었을 때 그 직선의 기울기가 바로 미분값입니다.

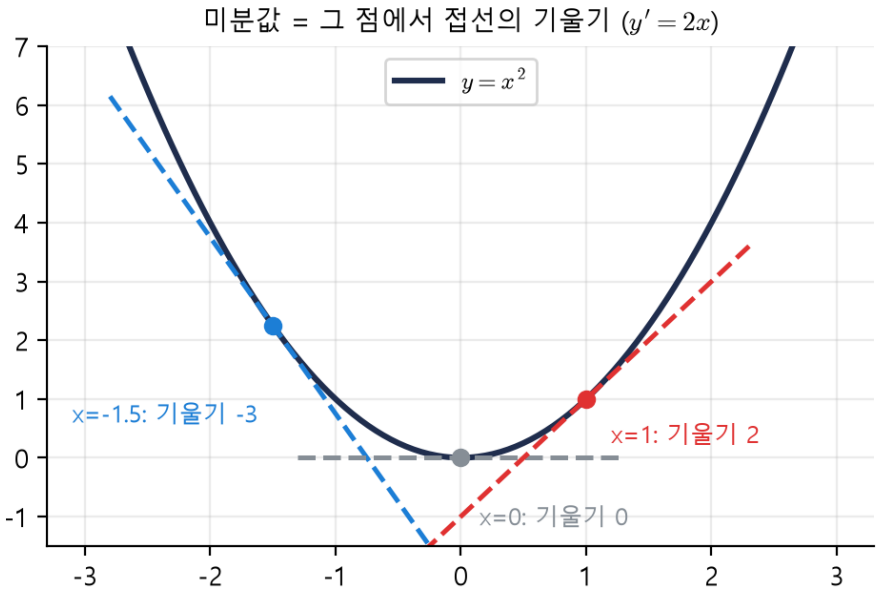


그림 7-1. y = x² 위의 세 점에서 그은 접선. 바닥(x=0)에서는 기울기가 0 이다

그림에서 바닥의 기울기는 0, 바닥 왼쪽은 음수, 오른쪽은 양수입니다. 그래서 "기울기가 0 인 곳 = 바닥(최솟값)"이고, "기울기의 반대 방향으로 가면 바닥에 가까워진다"는 사실을 얻습니다. 이것이 경사하강법의 핵심 아이디어입니다.

7.1 꼭 알아야 할 미분 규칙 3가지

미분한 결과는 y′ (와이 프라임) 또는 dy/dx 로 씁니다.

식	미분 결과	이유
$y = 3$	$y' = 0$	상수는 변하지 않으므로 변화량(기울기) 0
$y = 2x + 3$	$y' = 2$	2x = 2x¹ → 지수 1 이 앞으로 내려와 2·1·x^0 = 2, 상수 3 은 0
$y = x^2$	$y' = 2x$	지수 2 가 앞으로 내려오고, 지수는 1 줄어 x¹

세 규칙은 사실 하나의 공식으로 정리됩니다. **지수를 앞으로 내리고, 지수에서 1 을 뺀다**. 앞에 곱해진 상수는 그대로 따라옵니다. ($x^0 = 1$)

$$\frac{d}{dx} x^n = n x^{n-1} \qquad \frac{d}{dx}(c x^n) = c n x^{n-1} \qquad \frac{d}{dx} c = 0$$

또 더하기로 이어진 식은 항마다 따로 미분해서 더하면 됩니다. 예) $(x^2 + 2x + 3)' = 2x + 2$

실습 코드 7-1 - 숫자로 기울기 재 보기 (수치 미분)

```
def diff(func, x, h=1e-5):
    """중앙 차분으로 x 지점의 기울기(미분값)를 근사"""
    return (func(x + h) - func(x - h)) / (2 * h)

print("y = 3     , x=2 에서 기울기:", round(diff(lambda x: 3, 2), 4))
print("y = 2x + 3, x=2 에서 기울기:", round(diff(lambda x: 2*x + 3, 2), 4))
print("y = x^2   , x=2 에서 기울기:", round(diff(lambda x: x**2, 2), 4), " (2x = 4)")
print("y = x^2   , x=-1 에서 기울기:", round(diff(lambda x: x**2, -1), 4), " (2x = -2)")
```

실행 결과

```
y = 3     , x=2 에서 기울기: 0.0
y = 2x + 3, x=2 에서 기울기: 2.0
y = x^2   , x=2 에서 기울기: 4.0  (2x = 4)
y = x^2   , x=-1 에서 기울기: -2.0  (2x = -2)
```

실습 코드 7-2 - sympy 로 미분 규칙 확인

```
x = sp.symbols('x')

for expr in [sp.Integer(3), 2*x + 3, x**2, x**3, 5*x**4]:
    print(f"y = {str(expr):8s} ->  y' = {sp.diff(expr, x)}")
```

실행 결과

```
y = 3        ->  y' = 0
y = 2*x + 3  ->  y' = 2
y = x**2     ->  y' = 2*x
y = x**3     ->  y' = 3*x**2
y = 5*x**4   ->  y' = 20*x**3
```

핵심 포인트

- 미분 = 한 점에서의 기울기. 기울기 0 = 바닥.
- $x^n \to n \cdot x^{n-1}$, 상수 → 0, 더하기는 항별로.

확인 문제

Q1. 다음을 x 에 대해 미분하시오. (1) y = 7 (2) y = 5x - 4 (3) $y = 3x^2$ (4) $y = x^3 + x$

Q2. $y = x^2 - 4x$ 에서 기울기가 0 이 되는 x 는? (힌트: $y' = 0$)

CHAPTER 8

편미분 (Partial Derivative)

영상 구간 **08:20~09:47** 변수가 2개 이상일 때의 미분

학습 목표
- 편미분 기호 ∂ 를 읽을 수 있다.
- '나머지 변수는 상수 취급' 규칙으로 편미분을 계산할 수 있다.

지금까지는 변수가 x 하나였습니다. 그런데 머신러닝 모델은 w, b 처럼 변수가 여러 개입니다. 변수가 2개 이상인 식에서 **한 변수에 대해서만** 미분하는 것을 **편미분**이라고 합니다.

$$y = 2a^2 + 3b + 6$$

편미분은 d 대신 둥근 d 인 **∂** ("라운드" 또는 "파셜") 를 씁니다. ∂y/∂a 는 "y 를 a 에 대해 편미분"이라고 읽습니다.

규칙: 미분하는 변수를 제외한 나머지는 모두 상수 취급

a 에 대해 편미분	b 에 대해 편미분
$\frac{\partial y}{\partial a} = \frac{\partial}{\partial a}(2a^2 + 3b + 6) = 4a$	$\frac{\partial y}{\partial b} = \frac{\partial}{\partial b}(2a^2 + 3b + 6) = 3$
b 와 6 을 숫자처럼 보면 사라지고, 2a² → 4a	2a² 와 6 을 숫자처럼 보면 사라지고, 3b → 3

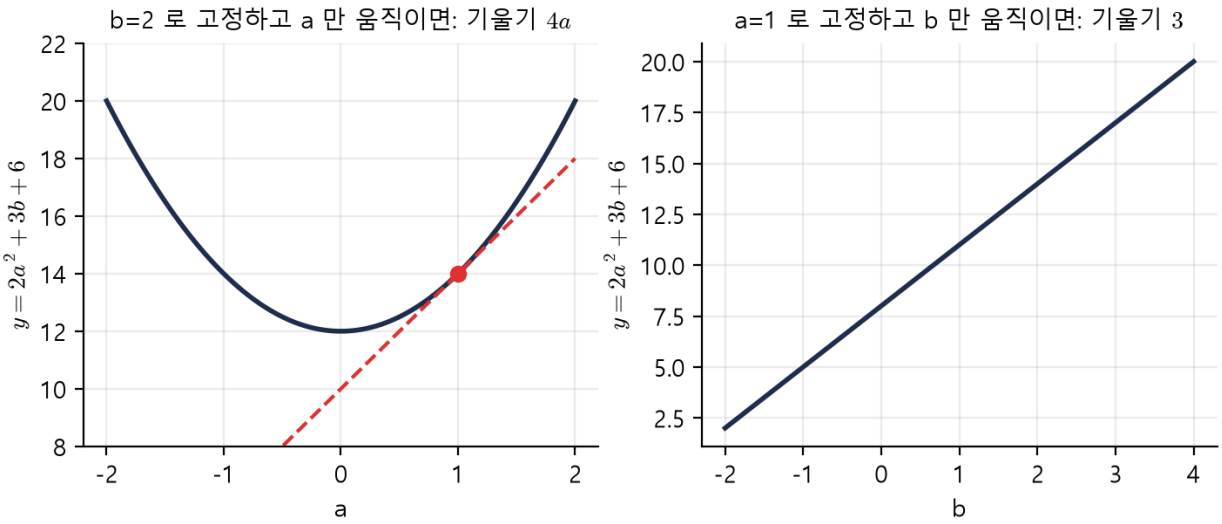


그림 8-1. 한 변수를 고정하고 다른 변수만 움직이면 보통의 1변수 미분과 같아진다

그림처럼 b 를 고정하면 y 는 a 에 대한 2차 함수, a 를 고정하면 b 에 대한 1차 함수가 됩니다. 편미분은 "**다른 축은 멈춰 두고 한 방향으로만 걸을 때의 기울기**"입니다. 모든 변수의 편미분을 모아 놓은 것을 **그래디언트(gradient, 기울기 벡터)**라고 부르며, 경사하강법의 이름이 여기서 나왔습니다.

실습 코드 8-1 - 편미분 확인

```
a, b = sp.symbols('a b')
y = 2*a**2 + 3*b + 6

print("y     =", y)
print("∂y/∂a =", sp.diff(y, a))      # b, 6 은 상수 취급 -> 사라짐
print("∂y/∂b =", sp.diff(y, b))      # 2a^2, 6 은 상수 취급 -> 사라짐

# 숫자로 확인: (a, b) = (1, 2) 에서 a 만 아주 조금 움직여 보기
fy = sp.lambdify((a, b), y)
h = 1e-5
print("수치 ∂y/∂a (a=1) :", round((fy(1 + h, 2) - fy(1 - h, 2)) / (2*h), 4))
print("수치 ∂y/∂b (b=2) :", round((fy(1, 2 + h) - fy(1, 2 - h)) / (2*h), 4))
```

실행 결과

```
y     = 2*a**2 + 3*b + 6
∂y/∂a = 4*a
∂y/∂b = 3
수치 ∂y/∂a (a=1) : 4.0
수치 ∂y/∂b (b=2) : 3.0
```

핵심 포인트

- ∂y/∂a: a 만 변수, 나머지는 상수 취급.
- 모든 변수의 편미분 묶음 = 그래디언트.

확인 문제

Q1. $z = 3x^2 + 2xy + y^2$ 일 때 ∂z/∂x 와 ∂z/∂y 를 구하시오. (힌트: 2xy 를 x 로 편미분하면 y 는 상수)
Q2. $L = (w - 2)^2 + 5b$ 일 때 ∂L/∂w 와 ∂L/∂b 를 구하시오.

CHAPTER 9

합성함수의 미분 (연쇄법칙)

영상 구간 **09:47~12:44** g(f(x)) 의 미분, 치환해서 겉 미분 × 속 미분

학습 목표
- 합성함수 g(f(x)) 의 의미를 이해한다.
- 연쇄법칙(chain rule)으로 합성함수를 미분할 수 있다.

9.1 합성함수란?

합성함수는 함수 안에 함수가 들어 있는 것입니다. f 의 출력이 g 의 입력으로 들어갑니다.

$$f(x) = ax^2 + b, \qquad g(x) = x^3 \quad \Longrightarrow \quad g(f(x)) = (ax^2 + b)^3$$

g 는 "넣은 것을 세제곱하는" 함수이므로, x 자리에 f(x) 를 통째로 넣으면 $(ax^2 + b)^3$ 가 됩니다.

9.2 연쇄법칙: 겉 미분 × 속 미분

$(ax^2 + b)^3$ 를 전개해서 미분할 수도 있지만 매우 번거롭습니다. 대신 안쪽을 한 글자로 **치환**합니다.

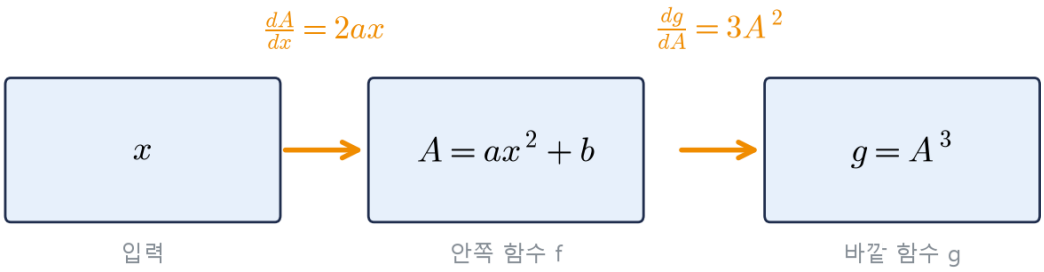


그림 9-1. 계산 흐름: x → A → g. 각 단계의 미분을 곱하면 전체 미분이 된다

① **치환**: $A = ax^2 + b$ 로 두면 $g = A^3$
② **겉 미분**: A^3 를 A 에 대해 미분 → $3A^2$
③ **속 미분**: $A = ax^2 + b$ 를 x 에 대해 미분 → $2ax$ (b 는 상수라 0)
④ **곱하고 되돌리기**: $3A^2 \cdot 2ax$ 에 $A = ax^2 + b$ 를 다시 넣는다

$$\frac{dg}{dx} = \frac{dg}{dA} \cdot \frac{dA}{dx} = 3A^2 \cdot A' = 3(ax^2 + b)^2 \cdot (2ax) = 6ax\,(ax^2 + b)^2$$

주의
- 가장 흔한 실수는 겉 미분 $3A^2$ 에서 멈추는 것입니다. **반드시 속 미분 A′ 를 곱해야** 합니다.
- 영상에서는 이 계산을 ∂/∂x 로 썼지만, 변수가 x 하나뿐이므로 보통은 d/dx 로 씁니다. 계산 방법과 결과는 같습니다.

실습 코드 9-1 - 연쇄법칙 결과가 맞는지 sympy 로 확인

```
a, b, x = sp.symbols('a b x')
f = a*x**2 + b          # 안쪽 함수 f(x)
g = f**3                # 바깥 함수 g(u) = u^3 에 f 를 넣은 g(f(x))

direct = sp.diff(g, x)                # sympy 가 바로 미분
chain  = 3*f**2 * sp.diff(f, x)       # 연쇄법칙: 바깥 미분 x 안쪽 미분

print("g(f(x))       =", g)
print("sympy 미분    =", direct)
print("연쇄법칙 결과 =", chain)
print("두 결과 같은가?", sp.simplify(direct - chain) == 0)
```

실행 결과

```
g(f(x))       = (a*x**2 + b)**3
sympy 미분    = 6*a*x*(a*x**2 + b)**2
연쇄법칙 결과 = 6*a*x*(a*x**2 + b)**2
두 결과 같은가? True
```

딥러닝은 층(layer)이 함수처럼 겹겹이 쌓인 거대한 합성함수입니다. 출력에서 입력 쪽으로 연쇄법칙을 반복 적용해 각 가중치의 미분을 구하는 방법이 바로 **역전파(backpropagation)**입니다.

핵심 포인트

- 합성함수의 미분 = 겉 미분 × 속 미분.
- 딥러닝의 역전파 = 연쇄법칙의 반복.

확인 문제

Q1. $y = (2x + 1)^3$ 을 x 에 대해 미분하시오.
Q2. $y = (x^2 - 3)^2$ 을 x 에 대해 미분하시오.
Q3. $y = (5 - x)^2$ 을 x 에 대해 미분하시오. (힌트: 속 미분이 -1)

CHAPTER 10

머신러닝에 연결하기 (보충)

영상 구간 **12:44~끝** "경사하강법·최소제곱법을 이해할 때 이 수식들이 나온다"

학습 목표
- 평균제곱오차(MSE)를 Σ 와 ŷ 로 쓸 수 있다.
- 편미분·연쇄법칙으로 MSE 의 그래디언트를 유도할 수 있다.
- 경사하강법으로 w, b 를 직접 학습시킬 수 있다.

영상 마지막에서 "앞으로 경사하강법이나 최소제곱법을 이해할 때 이런 수식이 많이 나온다"고 했습니다. 이 장에서는 1~9장의 도구를 **전부** 사용해 선형 회귀 모델을 학습시켜 봅니다.

10.1 손실 함수: 평균제곱오차(MSE)

모델 ŷ = wx + b (5장) 의 성능은 오차 y - ŷ (1장) 로 잽니다. 오차의 부호를 없애려고 제곱하고, 데이터 n 개에 대해 평균(4장)을 내면 **MSE(Mean Squared Error)**가 됩니다.

$$L(w,b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - (wx_i + b))^2$$

MSE 는 w (또는 b) 에 대해 2차 함수(6장)이므로 그릇 모양이고, 바닥이 가장 좋은 w, b 입니다.

10.2 그래디언트 유도: 편미분 + 연쇄법칙

L 은 w, b 두 변수의 함수이므로 편미분(8장)합니다. 괄호 안을 A = y_i - (wx_i + b) 로 치환하면 A^2 의 합성함수(9장)입니다.

단계	w 에 대해	b 에 대해
겉 미분 $(A^2)'$	2A	2A
속 미분 A′	∂A/∂w = -x_i (y_i, b 는 상수)	∂A/∂b = -1 (y_i, wx_i 는 상수)
곱하기	2(y_i - $\hat{y}_i$)·(-x_i)	2(y_i - $\hat{y}_i$)·(-1)

여기에 4장의 성질(상수 -2, 1/n 은 Σ 밖으로)을 적용하면 최종 그래디언트가 나옵니다.

$$\frac{\partial L}{\partial w} = -\frac{2}{n}\sum_{i=1}^{n} x_i\,(y_i - \hat{y}_i) \qquad \frac{\partial L}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)$$

실습 코드 10-1 - 데이터 1개의 제곱오차를 sympy 로 편미분

```
w, b, xi, yi = sp.symbols('w b x_i y_i')
loss_i = (yi - (w*xi + b))**2          # 데이터 한 개의 제곱 오차

print("∂/∂w :", sp.factor(sp.diff(loss_i, w)))
print("∂/∂b :", sp.factor(sp.diff(loss_i, b)))
```

실행 결과

```
∂/∂w : 2*x_i*(b + w*x_i - y_i)
∂/∂b : 2*(b + w*x_i - y_i)
```

sympy 결과 $2x_i(b + wx_i - y_i)$ 는 $-2x_i(y_i - \hat{y}_i)$ 와 같은 식입니다(부호를 괄호 안으로 넣은 것). 위 유도가 맞았습니다.

10.3 경사하강법 (Gradient Descent)

7장에서 "기울기의 반대 방향으로 가면 바닥에 가까워진다"고 했습니다. 그래서 그래디언트에 작은 수 η (학습률, learning rate) 를 곱해 **빼 주는** 일을 반복합니다.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \qquad\qquad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

실습 코드 10-2 - 경사하강법으로 y = 2x + 1 찾기

```
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([3, 5, 7, 9, 11], dtype=float)    # 정답 규칙: y = 2x + 1

w, b = 0.0, 0.0       # 아무 값에서나 출발
lr = 0.05             # 학습률: 한 번에 얼마나 움직일지

for step in range(1, 1001):
    Y_hat = w * X + b                  # 1) 예측 y hat
    err = Y - Y_hat                    # 2) 오차 y - y hat
    mse = np.mean(err ** 2)            # 3) 손실 MSE
    dw = -2 * np.mean(err * X)         # 4) ∂MSE/∂w
    db = -2 * np.mean(err)             #    ∂MSE/∂b
    w -= lr * dw                       # 5) 기울기 반대 방향으로 이동
    b -= lr * db
    if step in (1, 10, 100, 1000):
        print(f"step {step:4d} | MSE = {mse:8.4f} | w = {w:.4f}, b = {b:.4f}")
```

실행 결과

```
step    1 | MSE =  57.0000 | w = 2.5000, b = 0.7000
step   10 | MSE =   0.0230 | w = 2.0968, b = 0.6507
step  100 | MSE =   0.0011 | w = 2.0209, b = 0.9247
step 1000 | MSE =   0.0000 | w = 2.0000, b = 1.0000
```

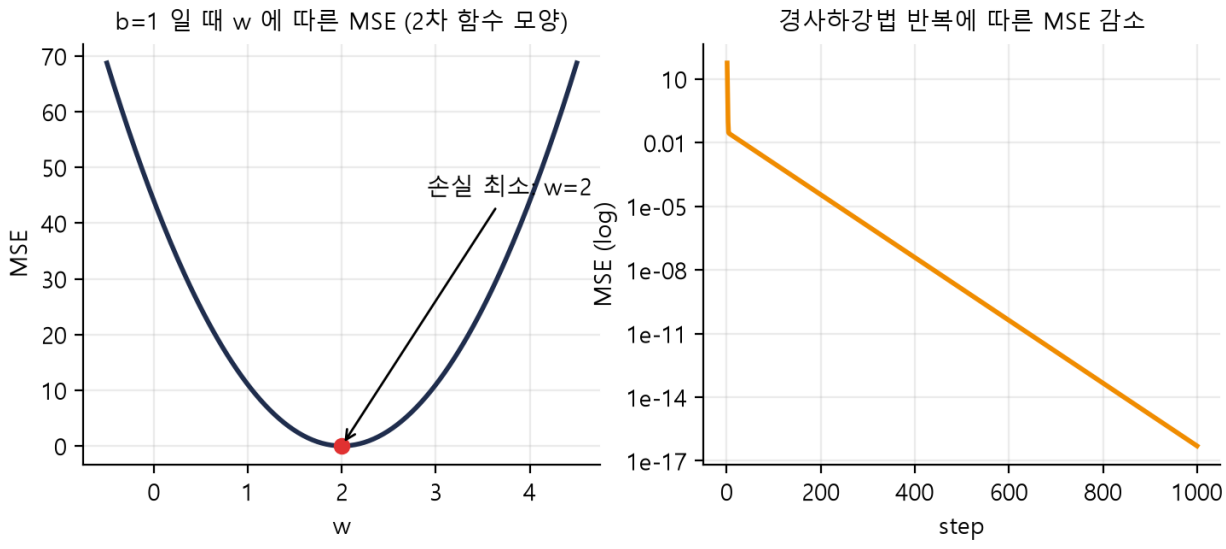


그림 10-1. 왼쪽: MSE 는 w 에 대한 그릇 모양. 오른쪽: 반복할수록 MSE 가 0 에 가까워진다

w = 0, b = 0 에서 출발했지만 1000 번 반복 후 정답 규칙 w = 2, b = 1 을 정확히 찾아냈습니다. (출력되는 MSE 는 그 step 에서 파라미터를 바꾸기 **전**의 값입니다.) 참고로 **최소제곱법(OLS)**은 반복하지 않고 $\partial L/\partial w = 0$, $\partial L/\partial b = 0$ 을 연립방정식으로 한 번에 풀어 바닥을 구하는 방법입니다.

정리: 이 장에서 쓴 영상의 도구들

사용한 개념	어디에 썼나
1장 y, ŷ	오차 $y_i - \hat{y}_i$
3·4장 Σ, 평균, Σ 성질	MSE 의 $(1/n)\Sigma$, 상수 -2 를 Σ 밖으로
5장 wx + b	모델 $\hat{y} = wx + b$
6장 2차 함수	MSE 는 그릇 모양 → 바닥이 정답
7장 미분	기울기의 반대 방향으로 이동
8장 편미분	w, b 각각의 기울기
9장 연쇄법칙	$(y - \hat{y})^2$ 를 미분할 때 속 미분 $-x_i$, -1

확인 문제

Q1. 학습률 η 를 너무 크게 하면 어떤 일이 생길 수 있을지 그림 10-1 왼쪽의 그릇 모양을 떠올리며 설명하시오.
Q2. 실습 코드 10-2 에서 lr = 0.05 를 0.01 로 바꾸면 1000 step 후 w, b 는 어떻게 될까? 직접 실행해 보고 이유를 적으시오.
Q3. 데이터 1개 (x, y) = (2, 5), 현재 w = 1, b = 0 일 때 $L = (y - \hat{y})^2$ 의 $\partial L/\partial w$ 와 $\partial L/\partial b$ 를 구하시오.

APPENDIX A

확인 문제 정답 및 해설

1장

Q1. $y = 170$, $\hat{y} = 168$, 오차 $y - \hat{y} = 2$ (cm)

Q2. $\bar{y} = (4 + 6 + 8) / 3 = 6$

2장

Q1. $f(4) = 3 \cdot 4 - 2 = 10$

Q2. $f(x) = 5x + 1$

3장

Q1. $1 + 3 + 5 + 7 = 16$, $i = 2 \sim 3$ 은 $x_2 + x_3 = 3 + 5 = 8$

Q2. $\Sigma\, i$ ($i = 1$ 부터 100 까지), 값은 5050

4장

Q1. $\bar{x} = 150 / 5 = 30$, $\Sigma x = 150$, $n \cdot \bar{x} = 5 \cdot 30 = 150$ 으로 성립

Q2. $8 \times 5 = 40$

Q3. $\bar{x} = 120 / 6 = 20$

5장

Q1. 가중치 $w = 3$, 편향 $b = 2$, $\hat{y} = 3 \cdot 4 + 2 = 14$

Q2. b 를 3 만큼 키운다 (w 는 그대로)

6장

Q1. $x = 0$ 일 때 최솟값 0

Q2. 제곱 부분이 0 이 되는 $x = 3$ 일 때 최솟값 1

7장

Q1. (1) 0 (2) 5 (3) $6x$ (4) $3x^2 + 1$

Q2. $y' = 2x - 4 = 0 \rightarrow x = 2$ (이 점이 그래프의 바닥)

8장

Q1. $\partial z/\partial x = 6x + 2y$, $\partial z/\partial y = 2x + 2y$

Q2. $\partial L/\partial w = 2(w - 2)$, $\partial L/\partial b = 5$

9장

Q1. $3(2x + 1)^2 \cdot 2 = 6(2x + 1)^2$

Q2. $2(x^2 - 3) \cdot 2x = 4x(x^2 - 3)$

Q3. $2(5 - x) \cdot (-1) = -2(5 - x) = 2x - 10$

10장

Q1. 한 번에 너무 멀리 움직여 바닥을 지나쳐 반대편 벽으로 튀고, 심하면 점점 더 높이 튀어 발산한다 (MSE 가 오히려 커짐).

Q2. 한 걸음이 작아져 1000 step 으로는 아직 바닥에 덜 도착한다. 실제로 실행하면 $w \approx 2.0039$, $b \approx 0.9860$ 으로 정답(2, 1)에 가까워지는 중이며, 반복 횟수를 늘리면 정답에 도달한다.

Q3. $\hat{y} = 1 \cdot 2 + 0 = 2$, 오차 $y - \hat{y} = 3$. $\partial L/\partial w = -2 \cdot x \cdot (y - \hat{y}) = -2 \cdot 2 \cdot 3 = -12$, $\partial L/\partial b = -2 \cdot (y - \hat{y}) = -6$

APPENDIX B

기호 · 공식 한눈에 보기

구분	기호 / 공식	읽는 법 · 뜻
기호	$y,\ \hat{y},\ \bar{y}$	실제값, 예측값(와이 햇), 평균값(와이 바)
함수	$y = f(x)$	x 를 넣으면 y 가 나오는 규칙 f
합계	$\sum_{i=1}^{n} x_i = x_1 + \cdots + x_n$	i 를 1 부터 n 까지 바꾸며 모두 더함
평균	$\bar{x} = \frac{1}{n}\sum x_i$	합계 ÷ 개수
Σ 성질	$\sum x = n\bar{x},\quad \sum b = nb,\quad \sum cx = c\sum x$	상수는 n 배 / 밖으로
1차 함수	$\hat{y} = wx + b$	w 가중치(기울기), b 편향(높낮이)
미분	$(x^n)' = nx^{n-1},\quad (c)' = 0$	한 점에서의 기울기
편미분	$\frac{\partial y}{\partial a}$	a 외의 변수는 상수 취급
연쇄법칙	$\frac{dg}{dx} = \frac{dg}{dA} \cdot \frac{dA}{dx}$	겉 미분 × 속 미분
MSE	$L = \frac{1}{n}\sum (y_i - \hat{y}_i)^2$	오차 제곱의 평균
경사하강법	$w \leftarrow w - \eta \frac{\partial L}{\partial w}$	기울기 반대 방향으로 조금씩 이동

참고

원본 영상: 테디노트 TeddyNote「#머신러닝 #딥러닝 입문자를 위한 초스피드 기초 #수학」
https://www.youtube.com/watch?v=vS51prw_yfw
이 교안은 영상의 흐름을 따라 정리하고, 그림·실습 코드·확인 문제와 10장(보충)을 덧붙였습니다.